MyCodeSchool

Code review — academy

Runde de review pe proiectul tău, cea mai nouă sus.

Proiect: mini-ERP de academie, C# consolă, fără framework. Regula fișierului: runda nouă se adaugă **sus**. Rundele vechi rămân dedesubt, condensate.

Runda 13 — 2026-11-03 — sha `3ae521c`

Scop: commit-ul `3ae521c` „collections exercises", adică cele 6 exerciții din `EXERCITII-COLECTII.pdf` și, înainte de ele, cele 3 🔴 din runda 12.

Metodă: `dotnet build`, apoi o copie în scratchpad. Înainte de fiecare drum am pus datele **din fișă** (commit-ul `7ad1f35`), nu cele din commit-ul tău (vezi M3). Am rulat fiecare „Gata când" cu inputul tastat, plus cazurile de margine cerute în fișă: `enrolments.txt` gol, un curs fără înscrieri, „te înscrii la Algebra, apoi din nou STATISTICI", un alt student logat. Fiecare 🔴 de mai jos e reprodus.

Poarta de compilare: ✅ **0 erori** (44 de warning-uri), al 9-lea commit verde la rând.

Bilanțul pe cele 6 exerciții

Ex.	Ce am rulat	Rezultat
1 `Dictionary<Guid, T>`	șterg `Cartea1`, apoi lista; prima linie din `courses.txt` copiată la final	✅ ieșirea din fișă; `Course "db7a391a-…" e deja in lista.`
2 fabricile, după tip	login Ștefan; linia `PROFESOR,…` în `users.txt`	✅ `Logat cu succes!` ; `Unknown user type: PROFESOR`
3 numărătorul	STATISTICI → 1	🔴 `Analiza Matematica cu 1 studenti` (fișa: `Algebra Matematica cu 2 studenti`) — B1 · cu `enrolments.txt` gol: `KeyNotFoundException` — B2
4 cursurile disponibile	CURSURI → 4; apoi după înscrierea la Algebra; apoi logat ca Ștefan Ognean	🔴 prima ieșire e corectă din noroc; după înscriere Algebra apare tot; la Ognean apare Algebra, la care e înscris — B3
5 pe departamente	STATISTICI → 2	✅ grupat și în ordine; 🟢 formatul rândului (C1)
6 studenți pe departament	STATISTICI → 3	🔴 `2 / 1 / 3` (fișa: `3 / 1 / 4`), fără `HashSet` și `UnionWith` — B4

🎉 Ce merge

Ce	Dovada
✅ **B1/r12: se iese din Cărți și din Cursuri**	`case 0: return;` în ambele; `0` → meniul principal → `0` → aplicația se închide
✅ **Ex. 1 complet, inclusiv partea grea**	`Read`, `Add` și `Remove` țin `items` și `itemsById` la fel, iar duplicatul la citire aruncă exact excepția cerută
✅ **Ex. 2 fără coduri scrise de mână**	cheile vin din `Type`-ul fabricilor
✅ **Ex. 5 cu `TryGetValue` + `out`**	sintaxa de care ne temeam e folosită corect: `if (d.TryGetValue(c.Department, out List<Course> courses)) courses.Add(c);`
`Course.ToString() => Name`	de-asta merge `String.Join(", ", d[key])` fără nicio buclă. Bine gândit!
Metodele vechi scoase	`StudentsCountForCourseId` și `GetStudentsCountForCourseId` nu mai există, cum cerea ex. 3

🔴 Critice

B1 — Numărătorul rămâne mereu la 1

`Enrolments/Repositories/EnrolmentRepository.cs:76`

CUM E ACUM: STATISTICI → 1 → `Analiza Matematica cu 1 studenti`. Algebra are 2 înscrieri, deci fișa cere `Algebra Matematica cu 2 studenti`.

DE CE: rândul este

```
d[enr.CourseId] = count++;
```

`count++` (post-increment) face două lucruri, în ordinea asta: **întoarce valoarea veche**, apoi crește variabila. Așa că în dicționar se scrie valoarea veche, adică 1, iar creșterea se pierde, pentru că `count` e o variabilă locală care dispare la următorul pas. Toate cursurile ajung la 1. La egalitate câștigă primul curs din fișier, adică Analiza. Cu `++count` (pre-increment) se scrie valoarea nouă.

FIX: `d[enr.CourseId] = count + 1;`. Varianta asta n-are capcana lui `++`.

B2 — Un curs fără înscrieri oprește aplicația

`Views/ViewStudent.cs:319, :321` (ex. 3) + `:356` (ex. 6)

CUM E ACUM: cu `Data/enrolments.txt` gol, STATISTICI → 1:

```
Unhandled exception. System.Collections.Generic.KeyNotFoundException: The given key
'db7a391a-ee78-4ae6-a3fb-a8c99a27fbf1' was not present in the dictionary.
```

La fel la STATISTICI → 3, de îndată ce un curs nu are nicio înscriere. Fișa cerea `Analiza Matematica cu 0 studenti`.

DE CE: numărătorul pune o cheie **doar** când întâlnește o înscriere. Un curs fără înscrieri nu apare deloc în dicționar, nici măcar cu 0. Iar `d[cheie]` la **citire** aruncă pentru o cheie lipsă: e exact `d["x"]` din dicționarul tău din `colections-csharp`. Fișa spunea: „Un curs fără nicio înscriere nu e în dicționar și are 0 studenți, fără excepție."

FIX: citești cu `TryGetValue`, care întoarce `false` și pune 0 în `out` pentru o cheie lipsă: `d.TryGetValue(c.Id, out int numar);`, apoi folosești `numar`. La fel la `:356`.

B3 — „Cursurile la care te mai poți înscrie" întreabă de ceilalți, nu de tine

`Enrolments/Repositories/EnrolmentRepository.cs:92` + `Views/ViewStudent.cs:173–181`

CUM E ACUM: pe datele din fișă, CURSURI → 4 dă exact cele 4 cursuri cerute. Dar:

- după ce te înscrii la **Algebra**, CURSURI → 4 o afișează **tot** pe Algebra;
- logat ca **Ștefan Ognean**, care e înscris doar la Algebra, lista o conține tocmai pe **Algebra**;
- după ce te înscrii la toate patru, nu apare `Sunteti inscris la toate cursurile`.

DE CE: condiția `if (!Guid.Equals(enr.StudentId, studentId))` adaugă cursul fiecărei înscrieri care **nu e a ta**. Rezultatul e „cursurile la care e înscris altcineva", nu „cursurile la care nu ești tu". Pe datele din fișă cele două mulțimi coincid din întâmplare: tu ești la Analiza și Algoritmica, ceilalți la celelalte 4. Un curs la care nu e înscris nimeni nu apare niciodată. Iar ordinea e cea a înscrierilor, nu cea din `courses.txt`.

Ce cerea fișa are doi pași: 1. **Mulțimea cursurilor TALE**: `CourseIdsForStudent` adaugă `enr.CourseId` când `enr.StudentId == studentId`. 2. **View-ul parcurge toate cursurile** (`courseService.GetCourses()`, în ordinea din fișier) și le afișează pe cele pentru care `!aleMele.Contains(c.Id)`. Ăsta e `Contains`-ul rapid pentru care am ales `HashSet`.

Dacă nu s-a afișat niciunul: `Sunteti inscris la toate cursurile`.

B4 — Ex. 6 adună numerele de la ex. 3, deci numără același student de două ori

`Views/ViewStudent.cs:346–360`

CUM E ACUM: STATISTICI → 3 → `Fizica: 2`, `Informatica: 1`, `Matematica: 3`. Fișa cere `3 / 1 / 4`. Partea mai importantă: chiar și cu B1 reparat, după ce te înscrii la Algebra, Matematica ar urca la 5. Fișa arată că trebuie să rămână 4 și te întreabă de ce.

DE CE: un departament nu are „suma studenților cursurilor lui", ci „câți studenți **diferiți** au măcar un curs acolo". Tu ești înscris la Analiza și, după test, și la Algebra: la sumă contezi de două ori, ca student al departamentului Matematica ești unul singur. O sumă nu poate ști că două numere conțin aceeași persoană. O mulțime poate, pentru că `Add` pe un element care există deja nu face nimic. De-asta fișa cerea `SortedDictionary<string, HashSet<Guid>>`: departamentul → mulțimea **Id-urilor** studenților, iar la final `Count`-ul mulțimii.

FIX (pașii din fișă): `StudentIdsForCourse(Guid courseId)` în repository + service; pentru fiecare departament o mulțime goală, apoi `departament.UnionWith(StudentIdsForCourse(c.Id))` pentru fiecare curs al lui. Rândul afișat: `Matematica: 4 studenti`.

B5 — B2/r12 neatins: un nume greșit la login închide aplicația cu stack trace

`Views/ViewLogIn.cs:40, :61, :75, :85`

CUM E ACUM: identic cu runda 12, `Xyz` / `Abc` la login:

```
Unhandled exception. System.ArgumentException: Userul nu aceste credentiale nu exista
   at stefan_academy_vanilla_charp.ViewLogIn.Logger() in ViewLogIn.cs:line 40
```

DE CE: mecanismul e explicat pe larg în runda 12: `Main` prinde `AcademyException`, iar `ArgumentException` nu e o `AcademyException`. Fișa de colecții începea cu „Înainte: cele 3 🔴 din runda 12". Din cele trei ai reparat una.

FIX: cele 4 `throw new ArgumentException` → `NotFoundException` / `ValidationException`.

B6 — B3/r12 neatins: după o eroare prinsă, aplicația se închide

`Views/ViewStudent.cs:214, :240, :282`

CUM E ACUM: modific `Cartea1` în `Carte!` → `Campul Numele nu e bun: contine caractere nepermise`, apoi programul se termină. La fel la a doua înscriere la Analiza.

DE CE: tot din runda 12: `catch`-urile din view sunt înguste (`DuplicateException`, `NotFoundException`), iar excepția urcă până în `Main`, în afara buclei de meniu. Execuția continuă după `catch`, iar în `Main` după `catch` nu mai urmează nimic.

FIX: `catch (AcademyException ex)` la `:214` și `:240`, plus `try` / `catch (AcademyException ex)` în jurul lui `CreateEnrolment` la `:282`.

🟡 Importante

- **M1 — decoratorul `TextMapperCuJurnal` a dispărut din `UserRepository`** (`UserRepository.cs:25`). Ai legat corect câmpul `path` de `Write` (jumătate din M3/r12 ✅), dar calea primită era tot `users.txt`. Cel mai probabil jurnalul a scris în fișierul cu utilizatori, așa că ai scos decoratorul. Acum clasa nu e folosită nicăieri, iar Decoratorul din rundele de design patterns s-a pierdut. Fix-ul era calea: `new TextMapperCuJurnal<User>(new UserTextMapper(), <calea spre jurnal.txt>)`.
- **M2 — verificarea nouă din `UpdateBook` e pe dos** (`BookService.cs:51`, M5/r12). Compară numele nou cu numele **aceleiași** cărți: `Cartea1` → `Cartea1` aruncă `Book "BookName" e deja in lista.`, iar `Cartea1` → `Alchimistul` trece și ajungi la **trei** cărți `Alchimistul`. Regula e „nicio

altă carte a studentului nu are deja numele ăsta": aceeași căutare ca la `AddBook`, pe textul tăiat. Iar cheia din excepție e numele cărții, nu textul `"BookName"`.

- **M3 — datele modificate sunt în commit.** `Cartea1` e șterasă din `books.txt`, înscrierea ta la Algebra e în `enrolments.txt`, iar `jurnal.txt` are o linie nouă (M2/r12: `jurnal.txt` nici nu trebuia să fie în git). Fiecare „Gata când" pornește de la datele din git. Cu datele tale, ex. 3 afișează altceva decât fișa, chiar și corect. Înainte de commit: `git checkout -- Data/books.txt Data/courses.txt Data/enrolments.txt Data/users.txt`.
- **M4 — `RASPUNSURI.txt` tot nu există** (M1/r12). Fișa de colecții mai adaugă două întrebări: ex. 1 (ce `Id` primește un curs creat cu `Course(name, department)`) și ex. 6 (de ce Algebra crește, iar Matematica nu). A doua e chiar B4.
- **M5 — restanțele din runda 12, neatinse:** `ModificareCarte` nu confirmă nimic (M4/r12, a 6-a rundă) · factory-urile nu verifică numărul de câmpuri (M6/r12).

🟢 Cleanups

- **C1 — rândurile din ex. 5 și 6 diferă de fișă:** `Departament: Fizica, cursuri: Optica, Mecanica` față de `Fizica: Optica, Mecanica`, iar la ex. 6 `Departament: Fizica, studenti: 2` față de `Fizica: 3 studenti`. Textul de meniu de la tasta 3 e „cati studenti sunt inscrisi in fiecare departament", iar fișa cere „cati studenti are fiecare departament". Fișa zice „rândul exact".
- **C2 — ex. 2: două căutări în loc de una** (`UserTextMapper.cs:47–49`): `ContainsKey`, apoi indexerul. `TryGetValue(cuv[0], out IUserFactory fabrica)` face una singură, ca în `FindById`. În constructor fiecare fabrică e creată de două ori (`new StudentFactory().Type`, apoi încă un `new StudentFactory()`): o creezi o dată, într-o variabilă.
- **C3 — `AfisareCursuriDisponibile`** (`ViewStudent.cs:179`): `GetCourse(id)` întoarce `null` pentru un curs șters, iar `c.Name` crapă. Dispare oricum odată cu B3, unde parcurgi cursurile, nu Id-urile.
- **C4 — un commit pentru toate cele 6 exerciții.** Fișa cerea „un exercițiu per commit", ca să se vadă ce a stricat ce.
- **C5 — restanțe din runda 12:** mesajele de validare cu spațiu în față (C1/r12) · `Guid`-uri în mesaje (C2/r12, se vede și la înscrierea dublă) · `ViewAdmin.loggedUser` (C4/r12).

Before / After

Unde	Acum	Cum ar trebui
`EnrolmentRepository.cs:76`	`d[enr.CourseId] = count++;`	`d[enr.CourseId] = count + 1;`
`ViewStudent.cs:319–321`, `:356`	`if (d[c.Id] > index)`	`d.TryGetValue(c.Id, out int numar);` `if (numar > index) { index = numar; course = c; }`
`EnrolmentRepository.cs:92`	`if (!Guid.Equals(enr.StudentId, studentId))`	`if (enr.StudentId == studentId)` (cursurile TALE)
`ViewStudent.cs:175–181`	`foreach (Guid id in h)` `Course c = courseService.GetCourse(id);`	`foreach (Course c in courseService.GetCourses())` `if (!aleMele.Contains(c.Id)) { ... }`
`ViewStudent.cs:352–357`	`sum += s[c.Id];`	`studenti.UnionWith(enrolmentService.GetStudentIdsForCourse(c.Id));` la final `studenti.Count`
`ViewLogIn.cs:40`	`throw new ArgumentException("...");`	`throw new NotFoundException("Userul", lastName + " " + firstName);`
`ViewStudent.cs:214`, `:240`	`catch (DuplicateException ex)`	`catch (AcademyException ex)`

Q&A — în `RASPUNSURI.txt`

1. **B1:** `int count = 1; int x = count++;` Cât e `x`, cât e `count`? Dar cu `++count`?
2. **B2:** `d[cheie]` și `d.TryGetValue(cheie, out v)` pe o cheie care lipsește: ce face fiecare? Când alegi una, când pe cealaltă?
3. **B4:** Ana e înscrisă la Analiza și la Algebra. Cât dă suma pe Matematica, cât dă `Count`-ul mulțimii? De ce?

▶ Ce urmează

Ordinea: **B1 → B2** (două rânduri, iar ex. 3 iese ca în fișă) → **B3** → **B4** → **B5, B6** (restanțele din runda 12) → M2, M3. Fiecare cu „Gata când"-ul lui din fișă, pornind de la datele din git, și câte un commit per exercițiu.

Runda 12 — 2026-10-14 — sha `c2b024b` — condensată

- ✅ **B1** `case 0: break` în Cărți/Cursuri, nu se mai putea ieși → **închis** (`case 0: return`)
- 🔴 **B2** nume greșit la login → stack trace (`ArgumentException` neprins) → **neatins** (B5/r13)
- 🔴 **B3** `catch` -uri înguste, aplicația se închide după o eroare prinsă → **neatins** (B6/r13)
- 🟡 M1 `RASPUNSURI.txt` → încă (M4/r13) · M2 `jurnal.txt` în git → încă (M3/r13) · M3 calea jurnalului → `path` legat ✅, dar decoratorul scos (M1/r13) · M4 confirmare la modificare → încă (M5/r13) · M5 carte dublă la modificare → verificare pe dos (M2/r13) · M6 a doua plasă → încă (M5/r13)
- 🟢 C1 mesaje · C2 `Guid` în mesaje · C4 `loggedUser` → încă (C5/r13) · C3 meniuri în gol la EOF → încă
- Q&A → **fără răspuns**

Runda 11 — 2026-10-07 — sha `e803498` — condensată

- ✅ **B1** `FindByStudentAndName` fără paranteze (method group) → **închis**, verificat prin rulare
- ⚠️ **B2** nume de carte invalid închidea aplicația → `Book` / `Course` convertite complet ✅, dar `catch` -urile din view tot înguste → **parțial** (B3/r12)
- ⚠️ **B3** înscriere dublă = stack trace → stack trace-ul a dispărut, dar sesiunea se termină → **parțial** (B3/r12)
- ⚠️ **B4** `continue` + `do-while(tasta != 0)` → ✅ în 3 meniuri · 🔴 în Cărți și Cursuri fix-ul a stricat ieșirea (B1/r12)
- ✅ **B5** `TryParse` ignorat la vârstă → **închis**
- 🟡 M1 `RASPUNSURI.txt` → încă (M1/r12) · M2 `jurnal.txt` în git → încă (M2/r12) · M3 calea jurnalului → parametru adăugat, nefolosit și greșit (M3/r12) · M4 confirmare la modificare → încă (M4/r12) · M5 a doua plasă → încă (M6/r12)
- 🟢 C1 mesaje → încă, extins la `Book` / `Course` (C1/r12) · C2 `Guid` în mesaj → încă (C2/r12) · C3 `loggedUser` → încă (C4/r12)
- 🔴 regresie nouă: schimbarea `catch` -ului din `Main` a lăsat login-ul fără plasă (B2/r12)
- Q&A → **fără răspuns**

Runda 10 — 2026-10-05 — sha `5f4704b` — condensată

- ✅ B2 `Update*` nu salvau → `Repository.Update` în 6 locuri, verificat după repornire
- ✅ B3 virgula în parolă → respinsă în `Admin` și `Teacher` · ⚠️ a doua plasă (număr de câmpuri, `ToText(null)`) nefăcută → M5/r11
- 🔴 B1 duplicat pe `Guid` nou → ✅ la cursuri și înscrieri · 🔴 **la cărți s-a stricat de tot** → B1/r11
- ✅ C1 · C2 · C3 · C5 · C6 (Strategy amânat, corect)
- 🟡 M1 `NotificatorConsola` fără drum → ✅ acum are drum (`ViewAdmin`) · M2 `jurnal.txt` în git → 🔴 încă (M2/r11) · M3 calea jurnalului → încă (M3/r11) · M4 stack trace la utilizator → ✅ `Main` afișează `.Message` · M5 `ModificareCarte` fără confirmare → încă (M4/r11)
- Q&A → **fără răspuns** (M1/r11)

Runda 9 — 2026-09-30 — sha `58681c9` — condensată

Observer funcționează (un `Add` , două reacții), zgomotul de la pornire a scăzut de la 29 de linii la 0, restanțele B3 și B4 din runda 7 au fost închise. Constatările:

- ✅ **B1** servicii construite de două ori în `ViewStudent` (inițializatori + constructor) → **închis** în runda 10.
- ✅ **B2** `ViewLogIn` abona un repository și pasa altul → **închis** (legătura). Notificatorul tot nu se aude, din alt motiv: M1, runda 10.
- ✅ **B3** `nameof(item)` scria literal „item” → **închis** (`typeof(T).Name`).
- ✅ **B4** `ThrowIfNull(nameof(interior))` nu declanșa niciodată → **închis**.
- ✅ **M1–M6** (`Repository<T> : INotificator` , `Display` mort, `typeof(UserTextMapper)` , `"\n"+Write` , typo-uri, newline final) → **toate închise**.
- ✅ **C1** vizibilități amestecate → **închis** · 🟡 **C2** calea jurnalului în 2 locuri → **încă deschis** (M3, runda 10).
- ⚠️ Q&A fără răspuns.

Runda 8 — 2026-09-29 — sha `067a141` — condensată

`TextMapperCuJurnal` = primul decorator adevărat al tău (aceeași interfață, referință la interior, deleagă întorcând rezultatul). Constatările:

- ✅ **B1** `nameof(T)` scria literal „T" → **închis** în runda 9 (`typeof(T).Name`).
- ✅ **B2** `new StreamWriter(cale)` trunchia, jurnal cu 1 intrare din 8 → **închis** (`, true`).
- ✅ **B3** ai schimbat toate cele 4 mappere deși fișa interzicea — dar te forțase B2 din runda 7 (`ITextMapper<T> : INotificator`); constrângerea era imposibil de respectat curat. **Închis** prin ruperea moștenirii.
- ✅ **B4** „repară linia, nu regula": `Console.WriteLine(Send(...))` scos doar la `UserTextMapper:72` → **închis**, toate cele 4 mappere curățate.
- 🟡 `return "";` inventat în `Send` → **încă prezent** în `Write` (M4, runda 9) · `Data/jurnal.txt` comis în git (fișier generat) → **încă în repo** · typo-uri → **M5** · 22 din 32 fișiere doar newline final normalizat de editor.
- **Restanțe runda 7:** B2 (Observer) ✅ închis în runda 9 · B3 (`DateTime.Parse` în Book) ✅ închis în runda 9, după 5 runde · B4 (`EnrolmentService` = `new()`) ✅ închis în `EnrolmentService` , 🔴 **reapărut de 3 ori în** `ViewStudent` (B1, runda 9).

Runda 7 — 2026-09-25 — sha `6bcc851`

Scop: verificat commit-ul `6bcc851` „factory pattern + decorator pattern + observer pattern" — adică tema din `EXERCITII-PATTERNS.pdf` . Metodă: `dotnet build` pe codul tău, apoi o copie în scratchpad pe care am **pornit efectiv aplicația** și am rulat drumul complet cerut de fișă (login → cărți → cursuri → statistici), plus testul de persistență și testul de 30 de secunde. Codul tău nu a fost modificat.

Poarta de compilare — TRECE

```
    33 Warning(s)
    0 Error(s)
```

Al treilea build verde consecutiv.

Bilanțul pe cele 6 cerințe din fișă

Cerință	Stare
0 — restanță: `BookTextMapper` să citească data ca `EnrolmentTextMapper`	🔴 **jumătate** — scrii corect, citești la fel ca înainte (B3)
1 — Factory Method peste `UserTextMapper.FromText`	✅ **FĂCUT, și făcut bine**
2 — Strategy / duplicarea din `Salary` și `Password`	🔴 **NEFĂCUT** — ai egalizat numerele, nu ai scos duplicarea (M3)
3 — Decorator peste `ITextMapper`	🔴 **NU EXISTĂ ÎN COD** (B1)
4 — Observer pe `Repository<T>`	🔴 **nu e Observer** (B2) — dar **rezultatul e atins**: se salvează pe disc
5 — provocarea, răspuns în scris	🔵 **nelivrat**

Ce merge — și e mai mult decât pare

Exercițiul 1 e făcut corect, în forma cerută. `IUserFactory` cu `Type` și `Create` , trei fabrici, array de fabrici, căutare după tip (`UserTextMapper.cs:65-72`). E fix forma din `ex10` , mutată pe cod adevărat. Nu am ce să-ți reproșez acolo.

Aplicația scrie acum pe disc — prima dată în șapte runde. Verificat prin rulare, nu prin citire:

```
adaugi „CarteDeTest" -> Repository: Item was added
inchizi, redeschizi, afisezi cartile:
   Nume carte: CarteDeTest , ID carte: 41a73ad9-... , adaugata in data: 9/25/2026
```

Și testul de 30 de secunde **trece**: pornire în care nu schimbi nimic → `Data/` neatins, toate patru fișierele cu același md5. Asta era „gata când"-ul exercițiului 4, și e îndeplinit. Mecanismul nu e Observer (vezi B2), dar problema pe care Observer-ul trebuia s-o rezolve — datele care dispăreau la

închidere — **e rezolvată**.

`User` a devenit `abstract` (`User.cs:5`) — corect, nimeni nu mai poate instanția un utilizator fără tip. Drumul complet login → cărți → cursuri → statistici rulează fără să crape.

🔴 Critice

B1 — Decoratorul din mesajul commit-ului nu există în cod

`Common/IDisplay.cs` , `Common/ISaver.cs` , `Common/Repository.cs:3`

CUM E ACUM: commit-ul se numește „factory pattern + **decorator pattern** + observer pattern". `grep -rn "Decorator" --include="*.cs" .` întoarce **zero** rezultate. Nu există nicio clasă care implementează `ITextMapper<T>` și primește un `ITextMapper<T>` în constructor.

Ce există în schimb: două interfețe noi, `IDisplay` și `ISaver` , implementate **direct** de `Repository<T>` :

```
public class Repository<T> : IDisplay, ISaver where T : class, IEntity
```

DE CE asta nu e Decorator: un decorator implementează aceeași interfață ca lucrul pe care-l învelește **și ține o referință la el**. Cele două lucruri împreună sunt tot pattern-ul: ești interschimbabil cu originalul (aceeași interfață) și poți delega (referința). `Repository<T>` nu învelește nimic — și-a adăugat două metode proprii. E ca și cum ai fi spus „fac un ambalaj" și ai fi scris metoda direct în cutie.

Testul din fișă nici nu se poate rula: „îl pui pe `UserRepository` fără să schimbi `Repository<T>` și fără să schimbi vreun mapper". Nu există ce să pui — iar `Repository<T>` a fost schimbat.

FIX: clasa cerută, forma din `ex11` : implementează `ITextMapper<T>` , primește un `ITextMapper<T>` în constructor, scrie în consolă, apoi cheamă interiorul și întoarce ce a întors el. Îl montezi în `UserRepository` , la locul unde azi scrie `new UserTextMapper()` .

B2 — Notificarea pleacă la CITIRE, nu la scriere. Și nu e Observer.

`Users/Mappers/UserTextMapper.cs:69` · `Books/Mappers/BookTextMapper.cs:18` · `Courses/Mappers/CourseTextMapper.cs:18` · `Enrolments/Mappers/EnrolmentTextMapper.cs:19`

CUM E ACUM — reprodus prin rulare. Pornești aplicația și, **înainte** să apuci să te loghezi, primești 40 de linii de zgomot:

```
UserTextMapper: user type STUDENT created succesfully
UserTextMapper: user type STUDENT created succesfully
   ... de 8 ori, exact cate randuri are users.txt ...
==================LOG IN==================

Numele: Prenumele: CourseTextMapper Course Analizacreated succesfully
BookTextMapper: Book created Cartea1 succesfully
   ... inca 30 de linii, peste promptul in care tocmai scrii ...
```

Promptul „Numele:" și „Prenumele:" ajung îngropate în mesaje. Utilizatorul scrie peste ele.

DE CE se întâmplă: ai pus `Send(...)` în `FromText` . Dar `FromText` nu e chemat când se creează ceva — e chemat din `Repository.Read()` , o dată pentru fiecare linie din fișier, la construirea repository-ului. Mesajul spune „created succesfully" despre șapte cărți care există de săptămâni pe disc. Nu s-a creat nimic; s-a citit.

Momentul în care chiar se creează ceva e `Repository.Add()` — și de acolo nu pleacă nicio notificare către mapper.

Și de ce construcția nu e Observer: ai scris `ITextMapper<T> : INotificator` (`Common/ITextMapper.cs:3`), deci fiecare mapper **este** propriul lui notificator, cu `Console.WriteLine` scris înăuntru. Într-un Observer, sursa ține o **listă** de ascultători și îi anunță pe toți, fără să știe cine sunt. Aici nu există listă, nimeni nu se abonează, și nu poți adăuga al doilea ascultător fără să deschizi mapperul și să-i rescrii metoda `Send` .

Fișa cerea explicit **doi** ascultători, și spunea de ce: „Cu unul singur n-ai cum să vezi diferența dintre Observer și un simplu apel la `Save()` pus în `Add`." Exact asta ai acum — un apel direct, plus un `Console.WriteLine` legat de altceva.

FIX: sursa e `Repository<T>`. `Add` și `Remove` sunt momentele. Ține o listă de `INotificator` și anunț-o din ele. Ascultătorul care salvează și ascultătorul care scrie în consolă sunt două clase separate, niciuna în mapper.

B3 — Restanța din exercițiul 0, închisă pe jumătate — a patra oară

`Books/Mappers/BookTextMapper.cs:19`

CUM E ACUM:

```
// BookTextMapper — SCRIE corect:
item.CreatedAt.ToString("yyyy-MM-dd")
// BookTextMapper — CITESTE ca inainte:
DateTime.Parse(cuv[3])

// EnrolmentTextMapper — forma corecta, pe care o ai deja scrisa:
DateTime.ParseExact(cuv[3], "yyyy-MM-dd", CultureInfo.InvariantCulture)
```

„Gata când"-ul era: *cele două mappere citesc data în același fel*. `grep -n DateTime` pe cele două fișiere arată că nu.

DE CE contează, deși pare cosmetic: `Parse` fără format întreabă **cultura mașinii** cum să citească textul. `ToString("yyyy-MM-dd")` scrie întotdeauna la fel. Deci ai o pereche asimetrică: scrii ISO, citești „ce-o înțelege calculatorul ăsta, azi". Pe laptopul tău merge. Pe altă mașină, `2025-12-03` poate fi citit altfel sau deloc — și atunci nu primești o eroare clară, ci o dată greșită salvată înapoi în fișier la prima modificare.

Se vede deja în rulare, la afișarea cărților:

```
Nume carte: Cartea1 , adaugata in data: 12/3/2025 12:00:00 AM
```

+ același tipar, a doua jumătate: la adăugare formatezi data (`ViewStudent.cs:189` , `ToString("dd-MM-yyyy")`), la afișarea listei nu. Aceeași regulă, două locuri, ai reparat unul.

E a patra oară când repari o clasă și îi lași fratele neatins — ex3, ex7, restanța din fișă, și acum jumătatea de citire a aceleiași clase pe care tocmai ai atins-o.

B4 — `EnrolmentService` citește fișierul de două ori

`Enrolments/Services/EnrolmentService.cs:10`

CUM E ACUM — reprodus prin numărare. Am numărat mesajele de încărcare de la pornire și le-am comparat cu numărul de rânduri din fișiere:

```
  users.txt       8 randuri ->  8 mesaje
  courses.txt     6 randuri ->  6 mesaje
  books.txt       7 randuri ->  7 mesaje
  enrolments.txt  8 randuri -> 16 mesaje     <- de doua ori
```

DE CE se întâmplă:

```
private readonly EnrolmentRepository repository = new();   // (1) initializator de camp

public EnrolmentService(EnrolmentRepository repository)
{
    this.repository = repository;                          // (2) corpul constructorului
}
```

Inițializatorii de câmp rulează **înaintea** corpului constructorului. Deci (1) construiește un repository — care în constructorul lui citește tot fișierul — iar (2) îl aruncă imediat și pune altul, care citește fișierul a doua oară.

`readonly` nu te apără aici, și e bine să știi de ce: `readonly` permite atribuirea în **ambele** locuri, inițializator și constructor. Interzice doar scrierea de mai târziu, din alte metode. Compilatorul n-are ce să-ți semnaleze — ai scris ceva perfect legal.

Rezultatul e o a doua copie a înrolărilor în memorie, de care nu știe nimeni. Azi doar irosește o citire. Mâine, când pui Observer-ul din exercițiul 4 pe repository, salvarea poate pleca de la instanța orfană — și scrii pe disc o listă din care lipsesc modificările.

Celelalte trei servicii nu au tiparul — `BookService`, `CourseService` și `UserService` declară câmpul fără `= new()`. E singurul loc, deci probabil o rămășiță.

FIX: șterge `= new()` de pe linia 10.

🟡 Importante

- **M1** — `UserTextMapper.cs:18, :31, :44` — `AdminFromText`, `TeacherFromText`, `StudentFromText`: **zero apeluri** în tot proiectul. Fabricile le-au înlocuit, dar au rămas în fișier. Curăță-le odată cu exercițiul care le-a făcut inutile.
- **M2** — `UserTextMapper.cs:53` — `ToText` are încă lanțul de `if (item is X)`. E oglinda switch-ului pe care tocmai l-ai desființat la citire. Fișa cerea doar `FromText`, deci nu e o greșeală — dar întrebarea rămâne: când adaugi `PARINTE`, câte fișiere atingi acum?
- **M3** — exercițiul 2 nu e făcut. `grep -rn "minim pe economie" Users` întoarce **2** rezultate (`Teacher.cs:36`, `Admin.cs:33`); „gata când”-ul cerea unul. Ai schimbat pragul lui `Admin` din 3500 în 4257, deci acum cele două settere sunt copie identică. Hint-ul 1 spunea că una dintre perechi e „aceeași regulă cu număr diferit” și cealaltă „exact același cod” — tu ai mutat-o pe prima în a doua categorie, fără să scoți duplicarea.
- **M4** — `UserService.cs:123` — `Load()` returnează `new User[10]`, adică zece `null`-uri, și nu e chemată de nimeni.
- **M5** — `ViewLogIn.cs:21` — garda nouă crapă pe EOF: `Unhandled exception. System.NullReferenceException at ViewLogIn.Logger() line 21`. `Console.ReadLine()` întoarce `null` când intrarea se termină, iar tu ceri `.Length`. Pe Enter gol funcționează corect — problema e doar la închiderea intrării (Ctrl+Z pe Windows).
- **M6** — 33 de warning-uri, aproape toate `CS8600` / `CS8602` / `CS8604`: „possible null reference”. M5 e fix unul dintre ele, devenit real. Merită citite, nu ascunse.

🟢 Cleanups

- **C1** — `CourseTextMapper.cs:18` — `"Course " + cuv[1] + "created succesfully"` → iese `Analizacreated`. Lipsește un spațiu.
- **C2** — formatele mesajelor diferă între mappere: `Name + ": "` la User și Book, `Name + " "` la Course și Enrolment.
- **C3** — `succesfully` → `successfully`, în toate patru.
- **C4** — `IDisplay` și `ISaver` sunt `internal`, `INotificator` e `public`. Alege una.
- **C5** — `Program.cs:18` — spațiu și indentare rămase după acolada de final.

Before / After

B4 — dublă citire

Acum	Corect
`private readonly EnrolmentRepository repository = new();`	`private readonly EnrolmentRepository repository;`
constructorul suprascrie, fișierul se citește de 2 ori	se citește o dată, cea injectată

B2 — unde se anunță

Acum	Corect
`FromText` cheamă `Send(...)` → anunț la fiecare linie citită de pe disc	`Add` / `Remove` anunță lista de ascultători
mapperul **este** notificatorul	mapperul nu știe că există ascultători
un singur ascultător, lipit în mapper	doi ascultători separați: unul salvează, unul scrie

B3 — citirea datei

Acum (`BookTextMapper`)	Corect (îl ai în `EnrolmentTextMapper`)
`DateTime.Parse(cuv[3])`	`DateTime.ParseExact(cuv[3], "yyyy-MM-dd", CultureInfo.InvariantCulture)`

Q&A — răspunde în scris

1. **B4:** de ce `readonly` nu a împiedicat ca acel câmp să fie scris de două ori? Ce anume interzice `readonly` , de fapt?
2. **B2:** ai un mapper care își scrie singur mesajul în consolă. Ce trebuie să atingi ca să adaugi un **al doilea** ascultător, care scrie în fișier? Și ce ar fi trebuit să atingi dacă era Observer?
3. **Restanța din fișă, exercițiul 1:** adaugi `PARINTE` cu un câmp în plus. Câte fișiere **existente** ești nevoit să modifici acum, cu fabricile? Câte erau cu `switch` -ul? Dacă răspunsul e același număr, spune-mi asta — e un rezultat corect.
4. **Exercițiul 5, rămas nelivrat:** după ce te loghezi și intri la cărți, câte copii ale lui `books.txt` ai în memorie? Ce se întâmplă cu Observer-ul din exercițiul 4 în situația asta?

Runda 6 — 2026-09-16 — sha `6839abf`

Scop: verificat commit-ul `6839abf` „fixes runda 5 + courses refactoring". Metodă: `dotnet build` pe codul tău, apoi o copie în scratchpad pe care am **pornit efectiv aplicația**, m-am logat, și am rulat un test de round-trip pe toate cele patru mapper-e (încarcă → salvează → compară fișierul). Codul tău nu a fost modificat.

Poarta de compilare — TRECE

```
35 Warning(s)
0 Error(s)
```

Al doilea build verde consecutiv. Regula a prins.

Runda cu cel mai bun bilanț de până acum: 5 din 7 critice închise

Runda 5	Stare azi
B1 — ai stricat `Books` cu prefixe în mapper fără prefix în fișier	✅ **ÎNCHIS**
B2 — `EnrolmentTextMapper` round-trip rupt din prima	✅ **ÎNCHIS**
B3 — `Save()` scrie `USER,` peste tot	⚠️ **jumătate** — `ToText` ramifică acum corect, dar efectul persistă din alt motiv (B1 de azi)
B4 — aplicația nu există, `Main` gol și `try` comentat	✅ **ÎNCHIS** — `Main` pornește `ViewLogIn`, `try`-ul e activ
B5 — din patru `Int32.Parse` ai șters trei	✅ **ÎNCHIS** — toate patru sunt `TryParse`
B6 — `UserTextMapper` nu ramifică pe tip (a patra oară)	🔴 **a CINCEA oară** — vezi B1
B7 — garda pe email doar la `CreateStudent`	✅ **ÎNCHIS** — e la toate trei

Ce s-a schimbat în felul în care lucrezi, și contează mai mult decât cifrele. B5 și B7 erau exact tiparul tău: repari linia pe care ți-o arăt, lași celelalte apariții ale aceleiași reguli. De data asta le-ai căutat singur pe toate. Al patrulea `Int32.Parse` de la `ViewStudent.cs:69` nu ți l-a mai arătat nimeni, și garda de email de la `CreateTeacher` / `CreateAdmin` nici atât. **Prima rundă în care ai aplicat regula, nu linia.**

`CourseRepository` e refactorizat corect, după același tipar ca `Books` și `Enrolments`, iar `CourseService` a scăzut de la 160 de linii la 40. Round-trip-ul pe `Books` și `Courses` e **OK**.

Ce a mers prost

Două lucruri, ambele găsite prin rulare, nu prin citire:

```
Books        round-trip: OK
Courses      round-trip: OK
Enrolments   round-trip: RUPT
Users        round-trip: RUPT
```

🔴 Critice

B1 — Nimeni nu se poate loga. A cincea rundă pentru același `if` lipsă

`Users/Mappers/UserTextMapper.cs:57-62`

CUM E ACUM: pornesc aplicația, introduc un utilizator care **există** în `users.txt`:

```
Numele: Scarlat
Prenumele: Stefan
System.ArgumentException: Error on user type
   at ViewLogIn.Logger() in ViewLogIn.cs:line 67
```

Aplicația te găsește, apoi nu știe ce ești. **Niciun login nu funcționează, deci nimic din aplicație nu e accesibil.** Și a doua față, demonstrată separat — o simplă încărcare urmată de salvare:

```
inainte: STUDENT,d383e75c-...,Stefan,Scarlat,stefanel.scarlat@gmail.com,18
dupa   : USER,d383e75c-...,Stefan,Scarlat,stefanel.scarlat@gmail.com,18
```

`users.txt` se corupe la prima salvare. Pentru un `Teacher` sau un `Admin` ar dispărea și salariul, orele și parola — câmpuri care nu există pe `User`, deci nu au unde să fie scrise.

DE CE: uită-te la cele două metode una lângă alta. `ToText` (linia 49) ramifică: `if (item is Student s) return StudentToText(s);`. Funcționează pentru că obiectul **există deja** și își poartă tipul cu el la execuție — `is` doar îl întreabă.

`FromText` nu are ce să întrebe. Ea **construiește** obiectul, deci în momentul apelului tipul încă nu există. Singurul loc unde mai e informația despre tip e chiar textul: `cuv[0]`, prefixul `STUDENT` / `TEACHER` / `ADMIN` pe care tu l-ai pus acolo. Linia 61 îl sare complet și construiește mereu `new User(...)`.

Asta e asimetria: **la scriere tipul e în obiect, la citire tipul e în date.** Dacă nu-l citești din date, se pierde definitiv — și tot ce urmează (`as Student`, `Save()`) lucrează cu un `User` gol.

Ai deja scrise `StudentFromText`, `TeacherFromText`, `AdminFromText` — liniile 14, 27, 40. **Toate trei sunt cod mort: nimeni nu le cheamă.** E al treilea tipar al tău, scrii piese pe care nu le conectezi, iar aici lipsește exact un `switch` pe `cuv[0]`.

Și un lucru care merită observat: bug-ul ăsta exista și în runda 5. Nu se vedea pentru că `Main` era gol. **Reparând B4, ai făcut vizibil B6.** Așa arată un proiect care începe să funcționeze — bug-urile ies la suprafață în ordinea în care ajungi la ele.

FIX: în `FromText`, ramifică pe `cuv[0]` și cheamă metodele pe care le ai deja.

B2 — Data se scrie altfel decât se citește. După o salvare, aplicația nu mai pornește

`Enrolments/Mappers/EnrolmentTextMapper.cs:10`

CUM E ACUM: încarc înscrierile și le salvez. Linia din fișier se transformă:

```
inainte: ...,db7a391a-ee78-4ae6-a3fb-a8c99a27fbf1,2025-10-20
dupa   : ...,db7a391a-ee78-4ae6-a3fb-a8c99a27fbf1,10/20/2025 12:00:00 AM
```

Apoi pornesc din nou aplicația, pe un calculator cu setări românești — adică al tău:

```
FormatException: String '10/20/2025 12:00:00 AM'
    was not recognized as a valid DateTime.
```

O salvare, și aplicația nu mai pornește niciodată.

DE CE: `item.CreatedAt` fără format cheamă `ToString()` implicit, iar acela folosește **cultura calculatorului pe care rulezi**. Pe o mașină americană iese `10/20/2025`, pe una românească `20.10.2025`. `DateTime.Parse` la citire face simetric: interpretează după aceeași setare. Deci formatul fișierului tău nu e o decizie a ta — e o consecință a setărilor din Windows.

De aici vine `FormatException`: șirul `10/20/2025` citit cu reguli românești cere **luna 20**, care nu există.

Compară cu geamănul lui, `Books/Mappers/BookTextMapper.cs:10`:

```
item.CreatedAt.ToString("yyyy-MM-dd")
```

Format fix, identic pe orice calculator. **L-ai reparat pe acesta chiar în runda asta** — și l-ai lăsat neatins pe celălalt, deși ai modificat `EnrolmentTextMapper` în același commit. Cele două fișiere au 19 linii fiecare și sunt aproape identice.

Ăsta e tiparul „repară linia, nu regula" în forma lui cea mai curată. Ai închis B5 și B7 căutând toate aparițiile — aici nu ai căutat.

FIX: `.ToString("yyyy-MM-dd")` la scriere. La citire, `DateTime.ParseExact(cuv[3], "yyyy-MM-dd", CultureInfo.InvariantCulture)` — ca să nu mai depindă de setări nici la intrare.

Before / After

B1 — `UserTextMapper.cs:57`

Acum:

```
public User FromText(string text)
{
    string[] cuv = text.Split(',');

    return new User(Guid.Parse(cuv[1]), cuv[2], cuv[3], cuv[4],
                    Int32.Parse(cuv[5]));
}
```

Cum ar trebui:

```
public User FromText(string text)
{
    string[] cuv = text.Split(',');

    switch (cuv[0])
    {
        case "STUDENT": return StudentFromText(text);
        case "TEACHER": return TeacherFromText(text);
        case "ADMIN":   return AdminFromText(text);
        default:
            throw new ArgumentException(
                "Tip de user necunoscut: " + cuv[0]);
    }
}
```

B2 — `EnrolmentTextMapper.cs:10` și `:17`

Acum:

```
return item.Id + "," + item.StudentId + ","
       + item.CourseId + "," + item.CreatedAt;

return new Enrolment(..., DateTime.Parse(cuv[3]));
```

Cum ar trebui:

```
return item.Id + "," + item.StudentId + "," + item.CourseId + ","
       + item.CreatedAt.ToString("yyyy-MM-dd");

return new Enrolment(..., DateTime.ParseExact(cuv[3], "yyyy-MM-dd",
                          CultureInfo.InvariantCulture));
```

Ordinea de lucru

1. **B1** — `switch` pe `cuv[0]`. E `if`-ul care îți deblochează toată aplicația.
2. **Pornește și loghează-te.** Abia acum poți vedea ecranul studentului, pe care l-ai scris în august și nu l-ai văzut niciodată rulând.
3. **B2** — formatul datei, în ambele sensuri.
4. **Rulează testul de round-trip pe toate patru** înainte de commit.

Q&A

1. `ToText` ramifică pe tip și funcționează. `FromText` are aceeași structură de date în față și nu poate face la fel. **Ce are una și nu are cealaltă, în momentul apelului?**
2. Ai avut build verde și în runda 5, și în runda 6. În amândouă, aplicația era inutilizabilă din motive diferite. **Ce test, rulat în 30 de secunde, ar fi prins ambele situații?**
3. Ai reparat data în `BookTextMapper` și ai lăsat-o în `EnrolmentTextMapper`, în același commit. La `Int32.Parse` și la garda de email ai făcut invers — le-ai căutat pe toate. **Ce a fost diferit? Cum ți-ai dat seama acolo că e o regulă, și aici nu?**

Runda 5 — 2026-09-15 — sha `eb5c7a8`

Primul build verde din nouă commit-uri. 7 constatări critice; starea fiecăreia e tabelată în runda 6, mai sus. Pe scurt: B1 (prefixe în `Books`) ✅ · B2 (round-trip `Enrolment`) ✅ · B3 (`Save()` scrie `USER,`) ⚠️ jumătate · B4 (aplicația nu pornea) ✅ · B5 (`Int32.Parse`) ✅ · B6 (`UserTextMapper` nu ramifică pe tip) 🔴 a cincea oară, închisă abia în `deab1cf` · B7 (garda pe email doar la `CreateStudent`) ✅.

Runda 4 — 2026-09-15 — sha `aba4e01`

Verificat commit-ul „code review fixes”. Buildul a picat a șasea oară — **1 eroare vizibilă, 6 ascunse în spatele ei**, pentru că `Enrolment` nu implementa `IEntity` și legarea tipului de bază oprea verificarea corpurilor. Prima rundă cu bilanț pozitiv pe restanțe (3 🔴 + 5 🟡 închise), dar aplicația a fost **golită** ca să treacă buildul.

#	Constatare	Stare azi
B1	nu compilează: `Enrolment` fără `IEntity` + 6 erori ascunse (apelanți neactualizați după mutare)	✅ **ÎNCHIS** — build verde, toți apelanții updatați
B2	aplicația nu mai există: `Main` gol, `Logger` comentat integral	🔴 NEATINS (R5 B4)
B3	`TryParse` lipit **sub** `Int32.Parse` , ×4 → meniul stricat și pentru input corect	⚠️ trei sferturi — `:69` rămas (R5 B5)
B4	`users.txt` a trecut la prefixe, `UserTextMapper` nu → contradicție garantată	⚠️ jumătate — fișierul se citește ✅, dar tot iese `User` (R5 B6)
B5	`Save()` rade salarii/parole și șterge prefixele	🔴 **înrăutățit** (R5 B3) — scrie `USER,` peste toate tipurile
B6	garda pe email doar la `CreateStudent`	🔴 NEATINS (R5 B7)
B7	`EnrolmentService` cu două liste paralele; `repository` nefolosit	✅ **ÎNCHIS**
M1	`EnrolmentRepository.FindById` ascunde metoda moștenită	🔴 NEATINS (R5 M2)
M2	`EnrolmentRepository` fără constructor `base(...)`	✅ ÎNCHIS
M3	`EnrolmentMapper` / `EnrolmentTextMapper` clase goale	✅ ÎNCHIS (dar round-trip rupt — R5 B2)
M4	`Email` : lungimea verificată înaintea lui `Trim`	🔴 NEATINS
M5	`DeleteUser` nu verifică dacă userul există	🔴 NEATINS
M6	nimic nu cheamă `Save()` pentru useri	🔴 **înrăutățit** — și `EnrolmentService` și-a pierdut `Save()`
M7	`InscriereCurs` ocolește serviciul	✅ ÎNCHIS
M8–M12	confirmare lipsă la `UpdateBook` , login tăcut, servicii auto-instanțiate, cod mort, naming DTO	⚠️ M10 jumătate; restul NEATINSE
C1–C8	`.gitignore` , cod comentat, `students.txt` , `aux` , parole, warnings, linie goală, newline la final	✅ C2 (`User.cs`); restul NEATINSE

Mecanismele explicate atunci și încă valabile: **o eroare de legare a unui tip suprimă erorile din tot codul care îl folosește** (B1) · `/* */` scoate codul din build, deci „build verde" devine un semnal fals (B2) · `Console.ReadLine()` **consumă** o linie la fiecare apel, deci două apeluri cer două linii (B3) · **tipul static al parametrului decide ce câmpuri sunt vizibile**, de-aia `ToText(User)` nu poate scrie un salariu (B5) · o constatare are un *unde* și o *regulă*; adresa e doar punctul din care ți-am arătat regula (B6).

Runda 3 — 2026-09-09 — sha `70360d6`

Verificat commit-ul „more user fixes". Buildul a picat cu 4 erori. Partea de fond s-a mișcat serios (B4 din R2 închis pe bune), dar `Student` / `Teacher` / `Admin` au primit `ITextMapper<T>` — șase metode pe care nimeni nu le putea chema.

#	Constatare	Stare azi
B1	nu compilează: `ViewLogIn:10`, `:21`, `Teacher.cs:87` — apelanți neactualizați după mutare	🔴 redeschis (R4 B1), același tipar
B2	aplicația nu pornește; după reparare, nici studenții nu se pot loga	🔴 NEATINS (R4 B4)
B3	`ITextMapper<Student>` nu e `ITextMapper<User>` → cele 6 metode noi, necheamabile; `Save()` rade parole	🔴 NEATINS (R4 B4, B5)
B4	fiecare `FromText` contrazice `ToText`-ul de deasupra (prefix + câmpuri lipsă)	✅ **ÎNCHIS** — round-trip verde pe toate trei
B5	`Admin` : `salary` înaintea lui `age` → admin de 10000 de ani cu salariul 48	✅ **ÎNCHIS**
B6	`if (FindById(student.Id) != null)` — mereu falsă, `Id` proaspăt generat	⚠️ o treime — `CreateStudent` ✅, `CreateTeacher` / `CreateAdmin` 🔴 (R4 B6)
B7	`Int32.Parse` pe tastă, 4 locuri	🔴 **înrăutățit** (R4 B3) — `TryParse` lipit sub `Parse`
B8	`Console.WriteLine` în loc de `throw` la `Salary` / `Password`	✅ **ÎNCHIS**, toate patru
M1	`UserRepository.FindById` ascunde metoda moștenită	✅ ÎNCHIS — refăcut în `EnrolmentRepository` (R4 M1)
M2	`Email` nu se curăță de spații	⚠️ jumătate — `email = text` ✅, lungimea tot pre-`Trim` (R4 M4)
M3	`UserMapper` nu e `static`	✅ ÎNCHIS
M4	`ToStudentUpdateResponse(User)` față de frații ei	✅ ÎNCHIS
M5	nimic nu cheamă `Save()` pentru useri	🔴 NEATINS (R4 M6)
M6	`DeleteUser` nu verifică dacă userul există	🔴 NEATINS (R4 M5)
M7	`StudentCreateRequest` în alt namespace	✅ ÎNCHIS
M8–M12	cod mort, `InscriereCurs`, confirmare lipsă, login tăcut, servicii duble	🔴 NEATINSE (R4 M7–M11)
C1–C9	cod comentat, `.gitignore`, `students.txt`, `aux`, parole, warnings, `using`-uri, newline, naming	✅ C7; C1 🔴 înrăutățit (+62 de linii); restul NEATINSE

Fix-urile complete, cu cod, pentru fiecare dintre cele de mai sus sunt în runda 4 (pentru cele încă deschise) — iar mecanismele explicate atunci rămân valabile: invarianța generică la B3, round-trip-ul la B4, „`int` e `int`" la B5, identitate vs. date la B6.

Runda 2 — 2026-09-09 — sha `3c1877d`

Verificat pasul 3 din Faza 1 (`User` → T0.5). Buildul a picat cu 16 erori (6 vizibile, 10 ascunse în spatele lor). Pașii 1 și 2 — `Course` și `Enrolment` — sări.

#	Constatare	Stare azi
B1	nu compilează: metode duplicate în `UserMapper` + DTO-uri șterse fără apelanți	🔴 redeschis (R3 B1)
B2	`UserTextMapper.FromText` face `Guid.Parse` pe prefixul de tip → aplicația nu pornește	🔴 NEATINS (R3 B2)
B3	`FromText` întoarce mereu `User` → adminul devine student, `Save()` rade parola și salariul	🔴 NEATINS (R3 B2, B3)
B4	`CreateUser` : `if (request != null)` mereu adevărat → orice cerere produce un `Student`	✅ ÎNCHIS
B5	`Salary` / `Password` în `Teacher` și `Admin` : `Console.WriteLine` în loc de `throw`	🔴 NEATINS (R3 B8)
B6	`Int32.Parse` pe tastă în `ViewStudent`	🔴 NEATINS (R3 B7)
M1	`UserRepository.FindById` ascunde metoda moștenită (`CS0108`)	🔴 NEATINS
M2	`Email` nu se face `Trim` , deși `FirstName` / `LastName` da	🔴 NEATINS
M3	`UserMapper` nu e `static` ; `ToTeacher` întoarce `User`	⚠️ jumătate
M4	ordinea argumentelor: `Teacher` greșit, `Admin` — capcană anunțată	⚠️ `Teacher` ✅ / `Admin` 🔴 (R3 B5)
M5	`StudentCreateRequest` în alt namespace decât frații lui	🔴 NEATINS
M6–M10	cod mort, `InscriereCurs` ocolește serviciul, modificarea cărții nu confirmă, login admin/profesor tăcut, servicii instanțiate de mai multe ori	🔴 NEATINS
C1–C7	cod comentat, `.gitignore` , `students.txt` mort, `Console.WriteLine(aux)` , parole în clar, warnings, `using` -uri	⚠️ C1 parțial; restul NEATINSE

Runda 1 — 2026-09-08 — sha `79abec3`

Verificat T0.0–T0.4 din `TASKS.md` . Buildul a trecut. **5/5 task-uri închise pe fond.**

#	Constatare	Stare azi
T0.0–T0.4	nu compilează · „Vezi cursurile" crapă · modificarea orfanizează cartea · `createdAt` ignorat · `CreateUser` face mereu un `User` gol	✅ toate 5 închise
B1	`Contains` la căutarea după nume → Enter pe gol înseamnă „primul din listă"	✅ ÎNCHIS
B2	`null` strecurat în lista de cursuri	✅ ÎNCHIS
B3	profesorul salvat nu mai poate fi citit (`ToText` ≠ constructorul din text)	🔴 reapărut de 3 ori în modele (R3 B4)
B4	`Int32.Parse` pe tastă	🔴 NEATINS de 2 runde (R3 B7)
M1	`Create` / `Update` din modele = cod mort	✅ șterse
M2	`BookUpdateRequest` cere o dată ignorată	✅ ÎNCHIS de T0.5
M3	`GetEnrolmentIdByStudentId` cod mort	🔴 NEATINS
M4	modificarea cărții nu confirmă nimic	🔴 NEATINS
M5	`User newUser = new();` inutil	✅ ÎNCHIS
M6	`InscriereCurs` ocolește serviciul	🔴 NEATINS
M7	login admin/profesor iese tăcut	🔴 NEATINS
M8	serviciile se instanțiază de mai multe ori	⚠️ `BookService` ✅, restul 🔴
C1–C7	`Console.WriteLine(aux)` · `.gitignore` · `students.txt` · parole în clar · warnings · `override` gol · `using`-uri	✅ C6; restul 🔴